The talk must work without wifi

Animated explainer · src/demo.js

A live agent needs a key, a model and a network. A talk needs none of those. demo.js is a fake model with the same shape as the real one, so --demo exercises every real seam without a single HTTP request.

8 scenes · 14 scripted steps · 7 sandbox checks · no network

CONTENTS

What is in this explainer

01 · Why this file exists 3

02 · The shape of a call 4

03 · Three acts 5

04 · Ordinary work 6

05 · The fence 7

06 · The scorecard 8

07 · The final answer 9

08 · Recap 10

Page numbers refer to the folio at the foot of each page.

01 · Why this file exists

No wifi. Same loop.

SRC/DEMO.JS · *source*

```
export function demoChat() {
  let i = 0;
  return async () => {
    const message = DEMO_SCRIPT[Math.min(i, DEMO_SCRIPT.length - 1)];
    i += 1;
    return {
      choices: [{ finish_reason: message.tool_calls ? 'tool_calls' : 'stop', message }],
    };
  };
}
```

Same shape — Returns choices[0].message.

Same loop — runTask cannot tell the difference.

Same guards — The sandbox is still real.

> **remember** A demo is credible when it runs the real program and only replaces the network.
> **On screen:** demo.js replaces the model, not the loop. The fake chat function has the same return shape. It walks a fixed script, then repeats the final answer so the loop can take its normal exit. Every tool, every hook, every sandbox refusal is still the real implementation.

02 · The shape of a call

A tool call is just JSON.

SRC/DEMO.JS · *source*

```
function call(id, name, args) {
  return {
    id,
    type: 'function',
    function: {
      name,
      arguments: JSON.stringify(args),
    },
  };
}
```

id — the matching key for the tool result.

type — always function in this demo.

name — the handler to dispatch.

arguments — a JSON string, exactly as a provider returns it.

> **On screen:** The fake model still has to produce the exact shape the dispatcher expects. A call has an id, a type, a function name, and JSON-encoded arguments. Keeping that shape means the demo never needs a special path through the loop.

03 · Three acts

The script has three acts.

Steps 1–5 — List, create, run, edit, run again.

Steps 6–12 — Path escape, home write, shell copy, force push, key echo, failing command.

Steps 13–14 — remember and recall - the only effects that outlive the process.

> The refusals are shown rather than described. The audience sees the guard hold.
> **On screen:** The first act is ordinary work: create a file, run it, edit it, run it again. The second act attacks the fence: writes outside the workspace, a shell copy, a force push, a key echo. The third act saves one durable memory and reads it back. That is the only effect that survives.

04 · Ordinary work

Look. Create. Run. Edit. Run again.

NODE SRC/CLI.JS --DEMO · *text*

```
think: Looking first: what is in the workspace...
tool → list_files {"path":"."}
result ← ... 1 folder(s), 2 file(s)

tool → create_file {"path":"demo/hello.js","content":"..."}
result ← created demo/hello.js (72 bytes)

tool → run_command {"command":"node demo/hello.js"}
result ← exit code 0
         ↳ stdout: Hello from a tiny agent

tool → edit_file {"path":"demo/hello.js", ...}
result ← edited demo/hello.js: 1 replacement(s), first at line 1

tool → run_command {"command":"node demo/hello.js"}
result ← exit code 0
```

> This is the loop an audience already understands: look, act, observe, repeat.
> **On screen:** The first five steps are the familiar agent story: list, create, run, edit, run again. Each tool call is real. create_file writes a real file; run_command runs a real shell. The demo is not a slideshow. It is the same loop with a scripted model.

05 · The fence

Seven attempts. Every guard visible.

- ⛔ Write ../escape.txt refused before touching disk.
- ✅ Read ../package.json allowed: reads are open.
- ⛔ Write ~/Desktop/pwned.txt refused as a home-folder write.
- ⛔ Shell copy to the Desktop refused: a command is not a way round the fence.
- ⛔ git push --force denied by policy, whatever CMD_POLICY says.
- 🔒 Echo the API key the child process sees a short env allow-list.
- 💥 A failing command returns the real exit code and an explanation.

> **the point** The demo proves the guards with the run's own tool results.
> **On screen:** The second act is a list of attacks, and each one produces an observation. Reads are open. Writes, deletes, shell copies, force pushes and key echoes are not. Nobody has to trust a slide: the refusals are in the transcript.

06 · The scorecard

Read the results back.

SRC/DEMO.JS · *source*

```
const DEMO_CHECKS = [
  { id: 's6', label: 'a write that escapes the workspace is refused',
    expect: /^FORBIDDEN: SAVE \.\.\/escape\.txt/ },
  { id: 's7', label: 'reads are open - one folder up works',
    expect: /^\.\.\/package\.json \(\d+ lines\)/ },
  { id: 's9', label: 'a shell copy out of the sandbox is refused',
    expect: /^FORBIDDEN: COMMAND COPY / },
  { id: 's11', label: 'the child process cannot see the API key',
    expect: /OPENROUTER_API_KEY=absent/ },
];
```

SANDBOX CHECKS · *text*

```
✓ a write that escapes the workspace is refused - FORBIDDEN: SAVE ../escape.txt
✓ reads are open - one folder up works - ../package.json (24 lines)
✓ a shell copy out of the sandbox is refused - FORBIDDEN: COMMAND COPY
✓ the child process cannot see the API key - OPENROUTER_API_KEY=absent
✓ 7/7 sandbox checks held
```

> **On screen:** The scorecard is not a separate claim. It reads the run's own tool messages. Each check has an id, a label and a regular expression that must match the result. The demo passes only when the guards really held. Otherwise the process exits non-zero.

07 · The final answer

Repeat the tail, take the exit.

SRC/DEMO.JS · *source*

```
const message = DEMO_SCRIPT[Math.min(i, DEMO_SCRIPT.length - 1)];
i += 1;
return {
  choices: [{ finish_reason: message.tool_calls ? 'tool_calls' : 'stop', message }],
};
```

Once the script is exhausted, the fake model keeps returning the final assistant message. The loop sees no tool calls and exits normally.

remember The demo does not need a special stop condition. It uses the same one as a real run.

On screen: After the last scripted step, the fake model does not throw or invent a new reply. It repeats the final message. That message has no tool calls, so the loop returns an answer. The demo exits through the same path as a real model. Nothing special is required.

08 · Recap

Fake the model. Keep the program.

• 14 scripted steps

• 7 guards checked

• 0 network calls

→

Same shape. The fake model returns the provider envelope.

→

Same guards. Tools and sandbox are untouched.

→

Same exit. No tool calls means an answer.

→

Check the proof. The scorecard reads the run back.

> **remember** An offline demo is honest when it exercises the real seams.
> **On screen:** Fourteen scripted steps, seven guard checks, zero network calls. The fake model has the real shape, so the real loop runs unchanged. That is how the demo stays honest when the conference wifi fails.